

Chapter 7 – Local and Global Variables

Slide 1 — What Are Local and Global Variables?

Definition

A **variable** is a named memory location used to store data.

Based on where a variable is declared, it can be **local** or **global**.

Local Variable

A **local variable** is declared inside a function or block. It can normally be used only within that area.

Global Variable

A **global variable** is declared outside all functions. It can be accessed by multiple functions in the program.

Analogy

Think about a college campus:

- **Local variable** → Your classroom notes. You mainly use them inside your classroom.
- **Global variable** → A notice board placed in the college. Different classrooms can access it.

Small Example

```
None
int college = 2026;    // Global

void show() {
    int marks = 90;    // Local
}
```

Here, `college` is global and `marks` is local.

Slide 2 — Why Are Local and Global Variables Used?

Why do we need them?

Variables have different purposes depending on where their data needs to be used.

Local Variables

Used when:

- Data is needed only for one function.
- We want to keep data limited to a specific task.
- We want to avoid accidentally changing the value elsewhere.

Global Variables

Used when:

- The same data is needed by multiple functions.
- A value needs to be shared across different parts of a program.
- The data represents information relevant to the whole program.

Important Statement

Use a local variable when the data belongs to one task; use a global variable when the data needs to be shared.

Base → Professional

Basic:

```
None
void calculate() {
    int total = 100;
}
```

`total` is needed only inside `calculate()`.

Professional idea:

Large programs may have shared information that several functions need to access, but unnecessary global variables can make programs harder to maintain.

Remember

Local → Limited use

Global → Shared use

Slide 3 — Scope: Where Can the Variable Be Used?

What is Scope?

Scope means the part of the program where a variable can be accessed.

Local Variable Scope

A local variable belongs to the function or block where it is declared.

None

```
void show() {  
    int marks = 90;  
    printf("%d", marks);  
}
```

Here, `marks` can be used inside `show()`.

Global Variable Scope

A global variable is declared outside functions.

None

```
int count = 10;  
  
void show() {  
    printf("%d", count);  
}
```

}

The function can access the global variable.

Important Statement

Scope determines where a variable is visible and usable.

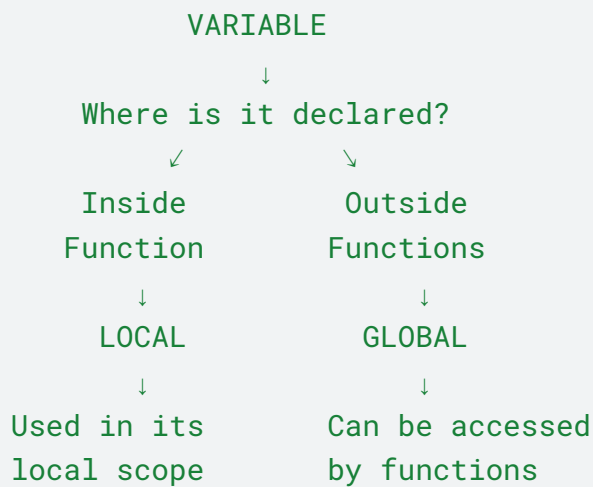
Simple Comparison

Local	Global
Declared inside a function/block	Declared outside functions
Limited scope	Wider scope
Usually used for specific tasks	Can be shared across functions

Slide 4 — Flowchart: Local vs Global

Variable Scope Flow

None



Example

None

```
#include <stdio.h>

int college = 2026;    // Global

void show() {
    int marks = 90;    // Local

    printf("%d\n", college);
    printf("%d", marks);
}
```

What happens?

- `college` is available to `show()`.
- `marks` belongs to `show()`.
- Another function cannot directly use `marks`.

Key Idea

Declaration location → Scope → Where the variable can be used

Slide 5 — Witty Recap

Imagine a College

Global Variable = College Notice Board

Everyone who needs the information can access it.

Local Variable = Your Personal Notebook

Only you use it for your particular work.

Practice Questions

1. Which statement correctly describes a global variable in C?

- A. It can be used only inside `main()`
- B. It is declared inside a function
- C. It is declared outside all functions
- D. It can be used only once

 **Hint:** Think about where the variable is declared.

Glossary:

Global — A variable declared outside all functions and available to different functions.

Answer: C. It is declared outside all functions

2.

None

```
#include <stdio.h>

int number = 10;

void display() {
    printf("%d ", number);
}

int main() {
    display();
    display();
    return 0;
}
```

What will be the output?

💡 **Hint:** `number` is declared outside the functions.

📖 **Glossary:**

Global variable — A variable that can be accessed by functions because it is declared outside them.

Answer:

None

10 10

3.

None

```
#include <stdio.h>

void show() {
    _____ marks = 90;
    printf("%d", marks);
}
```

Fill in the missing code.

💡 **Hint:** `marks` is a whole-number variable created inside the function.

📖 **Glossary:**

Local — A variable whose use is limited to the function or block where it is declared.

Answer: `int`

4. Arrange the words correctly:

values / parameters / receive / Functions / through

💡 **Hint:** Start with **Functions**.

📖 **Glossary:**

Parameter — A variable that receives a value when a function is called.

Answer: Functions receive values through parameters.

5. A function that **does not return a value** uses the keyword _____.

💡 **Hint:** It indicates that there is no return value.

📖 **Glossary:**

void — A keyword used when a function does not return a value.

Answer: void